Canny Edge detector Lecture 01

Concept 1. Convolution : A math operation on two function f and g, written $f*g$, (the integral of the product of two functions after one is reverse

$$(f*g)(t) = \int_{-\infty}^{\infty} f(\tau)g(t-\tau)\,d\tau$$

$$= \int_{-\infty}^{\infty} f(t-\tau)g(\tau)\,d\tau$$

widely used in signal processing, computer vision... statistics, some people call it stencil computing!

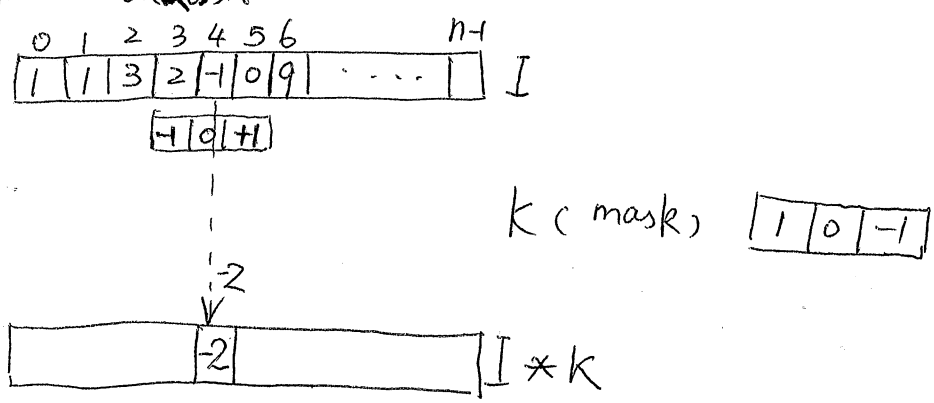
Simply put: If Input is I, and another function K (mask),

$I*K$ is implemented as following:

① flip-flop mask K, (right most element → left most)

② for I(i) in input, put mask on top of data with mask center aligned with data element to be processed(I(i)).

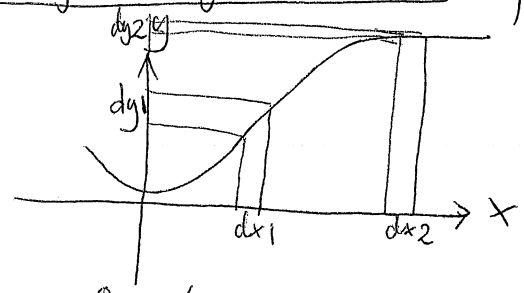
E.g.



$$2*(-1) + (-1)*0 + 0*(-1) = -2$$

③ Then allign the mask center with each element in input to compute result.

concept 2: gradient: general concept of derivation to the function, points to the direction of the greatest rate of increase of the function.



derivatives

$$f'(x) = \frac{df(x)}{dx} = \lim_{h \to 0} \frac{f(x) - f(x-h)}{h}$$

According to Talor expansion,

$$f(x_0 + h) = f(x_0) + hf'(x_0) + \frac{1}{2}h^2 f''(x_0) + o(h^3) \quad ①$$

$$f(x_0 - h) = f(x_0) - hf'(x_0) + \frac{1}{2}h^2 f''(x_0) + o(h^3) \quad ②$$

$$f'(x_0) \approx \frac{f(x_0+h) - f(x_0)}{h}$$ (The first order approximation)

Using equation ① subtracted by equation ②:

$$f'(x_0) = \frac{f(x_0+h) - f(x_0-h)}{2h} + o(h^2)$$ (Second order approximation)

For Images: $h=1$.

$$f(i) = \frac{f(i+1) - f(i-1)}{2}$$

$f * I$ (convolution): $\Rightarrow$ gradient

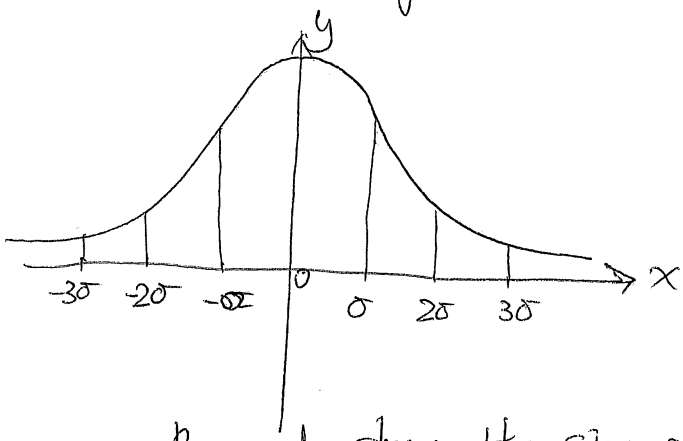
$$I = \frac{1}{2}[1, 0, -1]$$

Concept 3. Gaussian function: $e^{-\frac{x^2}{2\sigma^2}} = g(x)$

To do Gaussian Smoothing (filering)

We do convolution: $f(x) * g(x)$

How to generation the convolution mask using: ?



Research shows the size of guassian mask at least 5!

if $\sigma = 1$, a guassian mask with size of 5 is

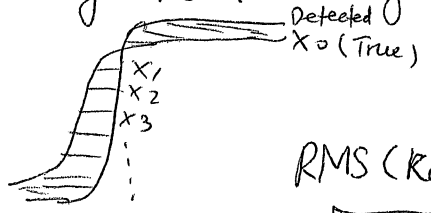
$$\left[e^{\frac{-4\sigma^2}{2\sigma^2}}, e^{-\frac{\sigma^2}{2\sigma^2}}, e^{0}, e^{-\frac{\sigma^2}{2\sigma^2}}, e^{\frac{-4\sigma^2}{2\sigma^2}} \right]$$

Canny Edge Detection Lecture 02.

It is optimal (Best one)

— Good detection: minimizing False positive & False Negative

— Good localization: minimizing RMS of distance of the detected Edge to True edge



RMS (Root Mean square)

$$= \sqrt{\frac{1}{n}\sum_{i=0}^{n} x_i^2}$$

— Single response

return one point for each true edge point.

Algorithm:

stage 1: Canny Enhancer:

① Gaussian Smoothing

② compute gradient $J(i, j) \Rightarrow \begin{cases} Jx(i,j) \rightarrow I * \begin{bmatrix} 1 & 0 & -1 \end{bmatrix} \\ Jy(i,j) \rightarrow I * \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix} \end{cases}$

③ compute edge strength:

$$e_s(i,j) = \sqrt{Jx(i,j)^2 + Jy(i,j)^2} \Rightarrow E_s$$

Range [0, 255]

④ Compute of Orientation of the edge normal

$$e_o(i,j) = \arctan\left(\frac{Jx(i,j)}{Jy(i,j)}\right) \Rightarrow E_o$$

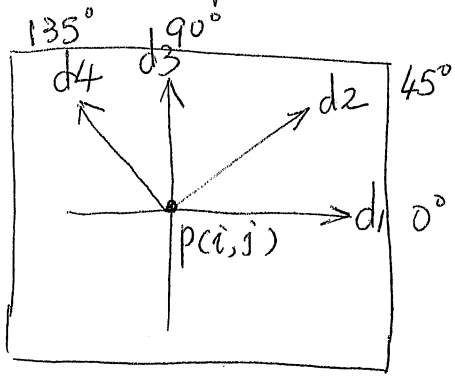
range $[0, \pi)$

Algorithm of Canny Edge Detector : 02

Stage 2: Non-Maxima Suppression (Thining the edge)

Input E_s, and E_o; Output I_n

for each pixel, we consider 4 directions of the edge normal.

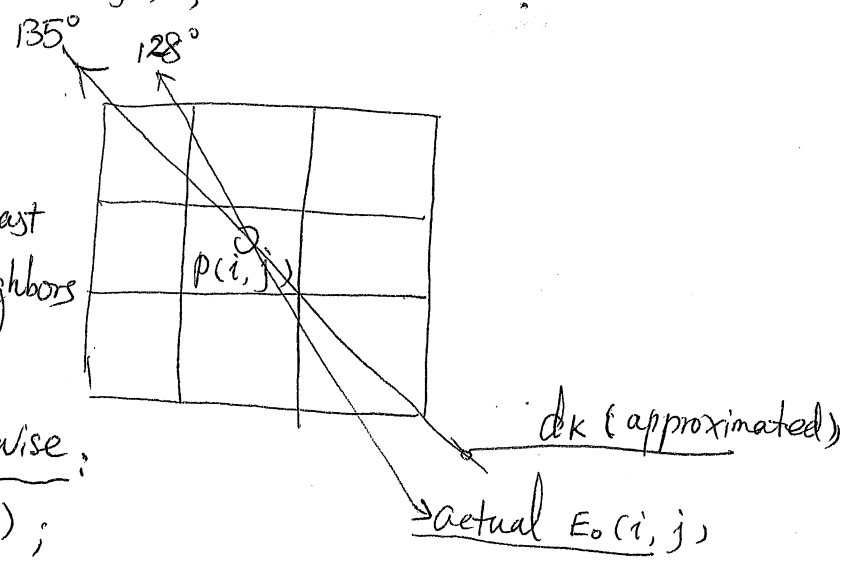


① We find the direction k that best approximate the Orientation of $E_o(i,j)$, called it d_k.

② If $e_s(i,j)$ is smaller than at least one of its two neighbors along d_k, then

$I_n(i,j) = 0$ otherwise;

$I_n(i,j) = E_s(i,j)$;



Stage 3: Threshholding: Using two threshholds T_{high} and T_{low}

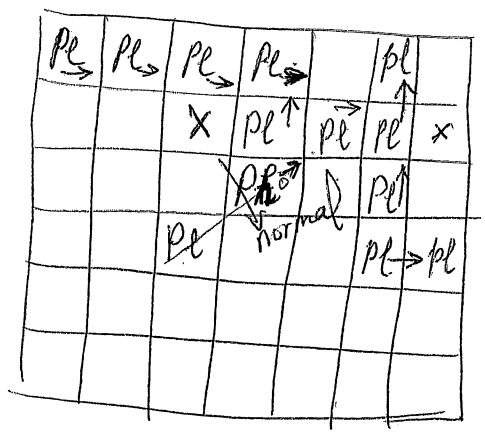
Input I_n (from previous stage) and E_o

- Scan I_n in a fixed order, I.E. Left → right, Top → Bottom
- Locate the next unvisited pixel such that $I_n(i,j) > T_{high}$;
- Start at this pixel, follow the chain of connected local maxima in both directions perpendicular to the edge normal as long as $I_n > T_{low}$. [edge Direction]
- Mark all visited points, keep a list of the locations of all points in the connected contour that has been found.

In diagram:

Ph means $P(i,j) > T_{high}$

Pl means $P(i,j) > T_{low}$ but $< T_{high}$



inside cells, "→" means the direction perpendicular to the edge normal [edge direction]

E_o is edge normal